

data_mining

April 20, 2026

1 Proyecto Implementacion fase 1

- Javier Prado 21486
- Bryan España 21550

1.1 Dataset a utilizar

El dataset a utilizar es sacado de: <https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset> Este es el conjunto de datos de evaluación de detección de intrusiones (CIC-IDS2017).

1.2 Construcción del dataset (CIC-IDS2017)

El dataset CIC-IDS2017 fue construido por el Canadian Institute for Cybersecurity (CIC) con el objetivo de crear un conjunto de datos realista y actualizado para la evaluación de sistemas de detección de intrusiones (IDS). A diferencia de datasets antiguos, este fue diseñado para simular el comportamiento real de una red empresarial bajo condiciones normales y bajo ataque.

1.3 Cómo fue construido

El dataset se generó en un entorno controlado de red durante un período de aproximadamente 5 días (lunes a viernes), donde se simuló el tráfico de una organización real. Durante este proceso:

Se configuró una infraestructura con múltiples máquinas (atacantes y víctimas) que representaban una red empresarial real. Se generó tráfico normal (benigno) utilizando perfiles de comportamiento de usuarios reales, simulando actividades como navegación web, envío de correos, uso de FTP y SSH. A partir del segundo día, se ejecutaron distintos tipos de ataques controlados en la red, incluyendo: Ataques de fuerza bruta (FTP y SSH) Ataques DoS y DDoS Escaneo de puertos Ataques web (SQL Injection, XSS) Botnets e infiltraciones Heartbleed

Todo el tráfico de red fue capturado en formato PCAP (packet capture), lo que representa los paquetes reales transmitidos en la red.

1.4 Procesamiento de los datos

Posteriormente, los datos capturados fueron procesados utilizando una herramienta llamada CFlowMeter, que transforma el tráfico de red en flujos (flows).

Un flow representa una comunicación entre dos dispositivos (IP origen y destino), y de cada flujo se extraen múltiples características como:

Número de paquetes Duración del flujo Tamaño de paquetes Flags TCP Estadísticas (media, desviación estándar, etc.)

Esto permitió generar alrededor de 80 características por flujo.

Características del conjunto de datos

Destination Port	Flow Duration	Total FwdPackets	Total Backward Packets
Total Length of Fwd Packets	Total Length of BwdPackets	Fwd Packet Length Max	Fwd Packet Length Min
Fwd Packet Length Mean	Fwd Packet Length Std	Bwd Packet Length Max	Bwd Packet Length Min
Bwd Packet Length Mean	Bwd Packet Length Std	Flow Bytes/s	Flow Packets/s
Flow IATMean	Flow IAT Std	Flow IAT Max	Flow IAT Min
Fwd IAT Total	Fwd IAT Mean	Fwd IAT Std	Fwd IAT Max
Fwd IAT Min	Bwd IAT Total	Bwd IAT Mean	Bwd IAT Std
Bwd IAT Max	Bwd IAT Min	Fwd PSHFlags	Bwd PSH Flags
Fwd URGFlags	Bwd URG Flags	Fwd Header Length	Bwd Header Length
Fwd Packets/s	Bwd Packets/s	Min Packet Length	Max Packet Length
Packet Length Mean	Packet Length Std	Packet Length Variance	FINFlag Count
SYNFlag Count	RSTFlag Count	PSH Flag Count	ACK Flag Count
URG Flag Count	CWEFlag Count	ECEFlag Count	Down/Up Ratio
Average Packet Size	Avg Fwd Segment Size	Avg Bwd Segment Size	Fwd Header Length
Fwd Avg Bytes/Bulk	Fwd Avg Packets/Bulk	Fwd Avg Bulk Rate	Bwd Avg Bytes/Bulk
Bwd Avg Packets/Bulk	Bwd Avg Bulk Rate	Subflow Fwd Packets	Subflow Fwd Bytes
Subflow Bwd Packets	Subflow Bwd Bytes	Init_Win_bytes_forward	Init_Win_bytes_backward
act_data_pkt_fwd	min_seg_size_forward	Active Mean	Active Std
Active Max	Active Min	Idle Mean	Idle Std
Idle Max	Idle Min		

1.5 Labels

Cada flujo fue etiquetado como:

Benigno (normal) Ataque (con tipo específico)

En total, el dataset contiene múltiples clases de ataque además de la clase normal, lo que lo hace ideal para problemas de clasificación supervisada en Machine Learning.

2 Analisis exploratorio

```
[1]: # Para descargar el dataset se debe descargar con el siguiente codigo
# se mantendra comentada esta seccion para evitar descargar el dataset cada vez
# que se ejecute el codigo

# al ejecutarlo se creara una carpeta en la carpeta del repo, dentro de esta
# carpeta se descargara el dataset
from pathlib import Path
import kagglehub
import shutil

# Ruta base (donde está el notebook)
BASE_DIR = Path().resolve()

# Carpeta dataset dentro del proyecto
DATASET_DIR = BASE_DIR / "dataset"
```

```

DATASET_HANDLE = "chethuhn/network-intrusion-dataset"

# Crear carpeta si no existe
DATASET_DIR.mkdir(parents=True, exist_ok=True)

# Evitar descargar si ya existe contenido
if any(DATASET_DIR.iterdir()):
    print(" Dataset ya existe en:", DATASET_DIR)
else:
    print(" Descargando dataset...")

    # Descargar dataset con el identificador completo de Kaggle
    download_path = Path(kagglehub.dataset_download(DATASET_HANDLE))

    # Copiar archivos desde la caché de Kaggle al proyecto
    for source in download_path.iterdir():
        destination = DATASET_DIR / source.name
        if source.is_dir():
            shutil.copytree(source, destination, dirs_exist_ok=True)
        else:
            shutil.copy2(source, destination)

    print(" Dataset descargado en:", DATASET_DIR)

```

Dataset ya existe en: C:\Users\javil\OneDrive\Documentos\U\Data Science\Intrusion_Detection_System\dataset

C:\Users\javil\OneDrive\Documentos\U\Data Science\Intrusion_Detection_System\myenv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

```
from .autonotebook import tqdm as notebook_tqdm
```

2.1 Analisis exploratorio detallado

Este bloque construye un EDA orientado a describir el dataset **CIC-IDS2017** desde cuatro angulos:

- estructura general del conjunto de datos
- distribucion de clases y archivos
- calidad de datos y variables redundantes
- características clave mas informativas para distinguir trafico benigno y ataques

```

[2]: from cicids_eda import run_eda

artifacts = run_eda(DATASET_DIR)

```

Construyendo vista general del dataset...

Evaluando calidad de datos...

Generando muestra balanceada para visualizacion y ranking de variables...

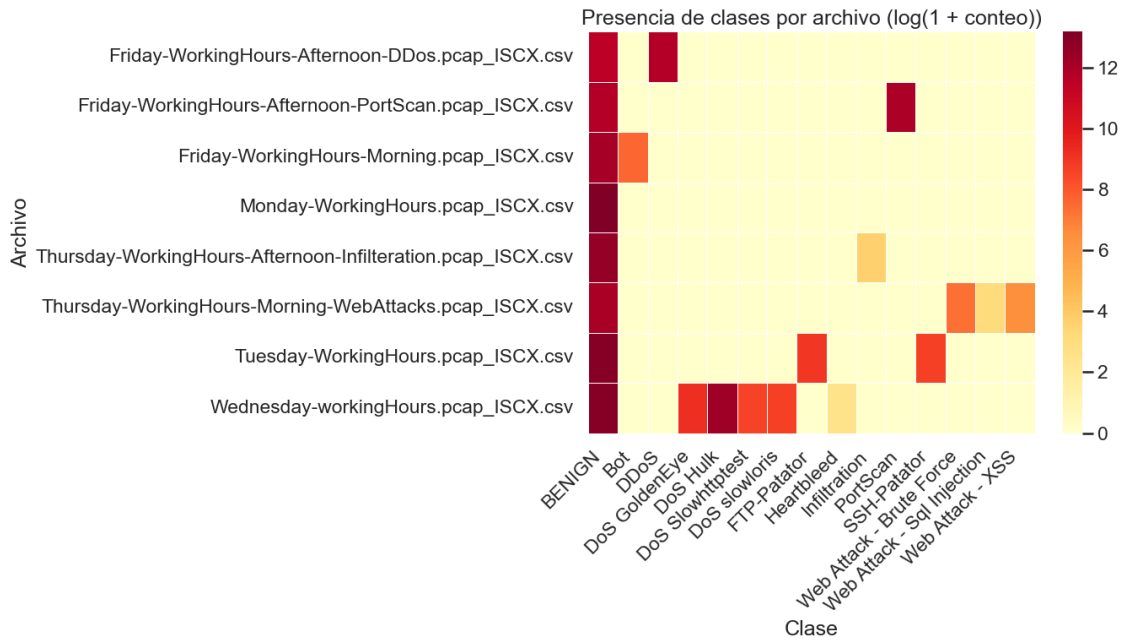
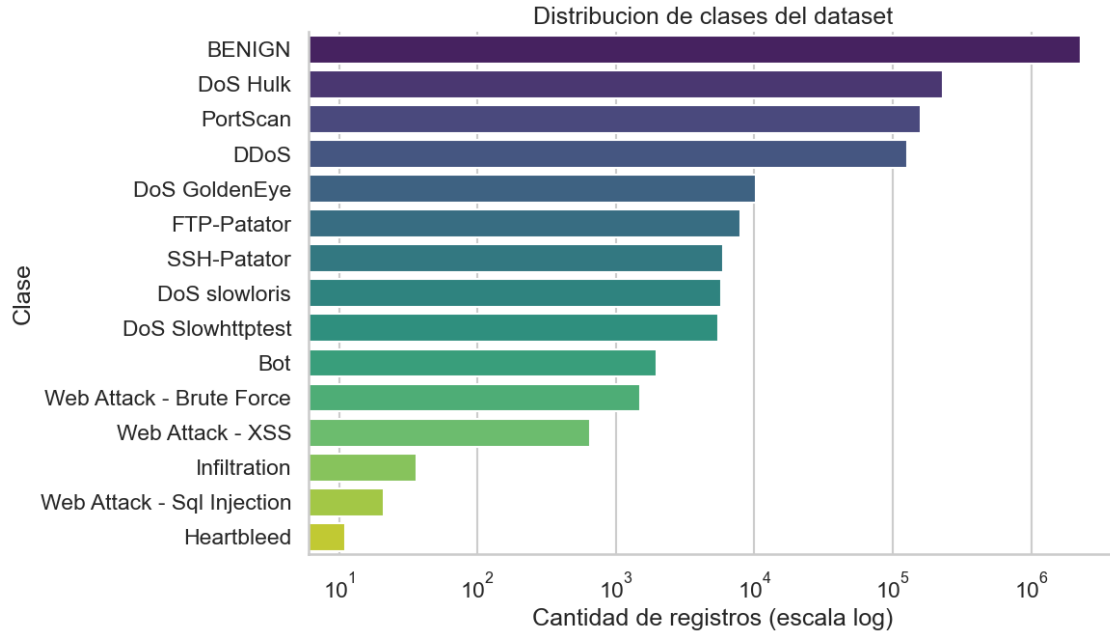
2.2 1. Vista general del dataset

	metric	value
0	Archivos CSV	8
1	Registros totales	2830743
2	Caracteristicas numericas y de red	78
3	Clases de trafico	15

	file	rows	classes \
0	Wednesday-workingHours.pcap_ISCX.csv	692703	6
1	Monday-WorkingHours.pcap_ISCX.csv	529918	1
2	Tuesday-WorkingHours.pcap_ISCX.csv	445909	3
3	Thursday-WorkingHours-Afternoon-Infiltration...	288602	2
4	Friday-WorkingHours-Afternoon-PortScan.pcap_IS...	286467	2
5	Friday-WorkingHours-Afternoon-DDoS.pcap_ISCX.csv	225745	2
6	Friday-WorkingHours-Morning.pcap_ISCX.csv	191033	2
7	Thursday-WorkingHours-Morning-WebAttacks.pcap_...	170366	4

	benign_pct	attack_pct	dominant_attack	dominant_attack_rows
0	63.52	36.48	DoS Hulk	231073
1	100.00	0.00	None	0
2	96.90	3.10	FTP-Patator	7938
3	99.99	0.01	Infiltration	36
4	44.52	55.48	PortScan	158930
5	43.29	56.71	DDoS	128027
6	98.97	1.03	Bot	1966
7	98.72	1.28	Web Attack - Brute Force	1507

	label	count	pct
0	BENIGN	2273097	80.3004
1	DoS Hulk	231073	8.1630
2	PortScan	158930	5.6144
3	DDoS	128027	4.5227
4	DoS GoldenEye	10293	0.3636
5	FTP-Patator	7938	0.2804
6	SSH-Patator	5897	0.2083
7	DoS slowloris	5796	0.2048
8	DoS Slowhttptest	5499	0.1943
9	Bot	1966	0.0695
10	Web Attack - Brute Force	1507	0.0532
11	Web Attack - XSS	652	0.0230
12	Infiltration	36	0.0013
13	Web Attack - Sql Injection	21	0.0007
14	Heartbleed	11	0.0004



2.3 2. Calidad de datos y variables redundantes

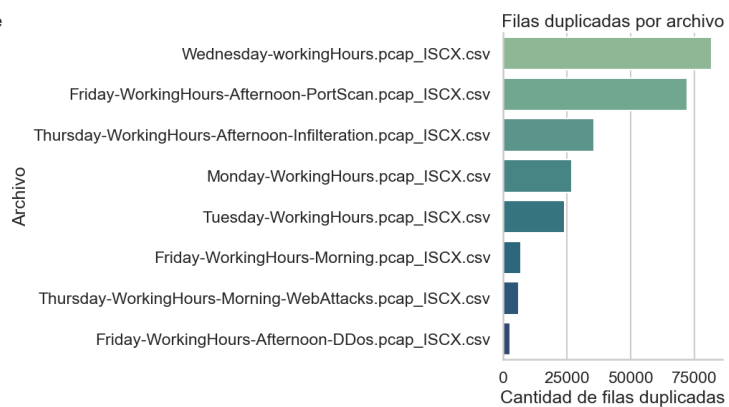
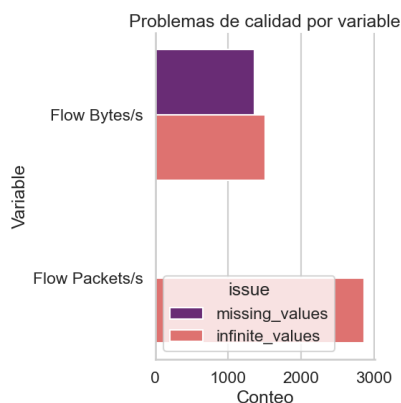
	feature	missing_values	infinite_values
0	Flow Packets/s	0	2867
1	Flow Bytes/s	1358	1509

	file	duplicate_rows	\
0	Wednesday-workingHours.pcap_ISCX.csv	81909	
1	Friday-WorkingHours-Afternoon-PortScan.pcap_IS...	72353	
2	Thursday-WorkingHours-Afternoon-Infiltration...	35630	
3	Monday-WorkingHours.pcap_ISCX.csv	26935	
4	Tuesday-WorkingHours.pcap_ISCX.csv	24065	
5	Friday-WorkingHours-Morning.pcap_ISCX.csv	6888	
6	Thursday-WorkingHours-Morning-WebAttacks.pcap_...	6066	
7	Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv	2633	

	duplicate_pct
0	11.82
1	25.26
2	12.35
3	5.08
4	5.40
5	3.61
6	3.56
7	1.17

	constant_feature
0	Bwd PSH Flags
1	Bwd URG Flags
2	Fwd Avg Bytes/Bulk
3	Fwd Avg Packets/Bulk
4	Fwd Avg Bulk Rate
5	Bwd Avg Bytes/Bulk
6	Bwd Avg Packets/Bulk
7	Bwd Avg Bulk Rate

	feature_a	feature_b
0	Total Fwd Packets	Subflow Fwd Packets
1	Total Backward Packets	Subflow Bwd Packets
2	Fwd Header Length	Fwd Header Length.1



2.4 3. Características clave del tráfico

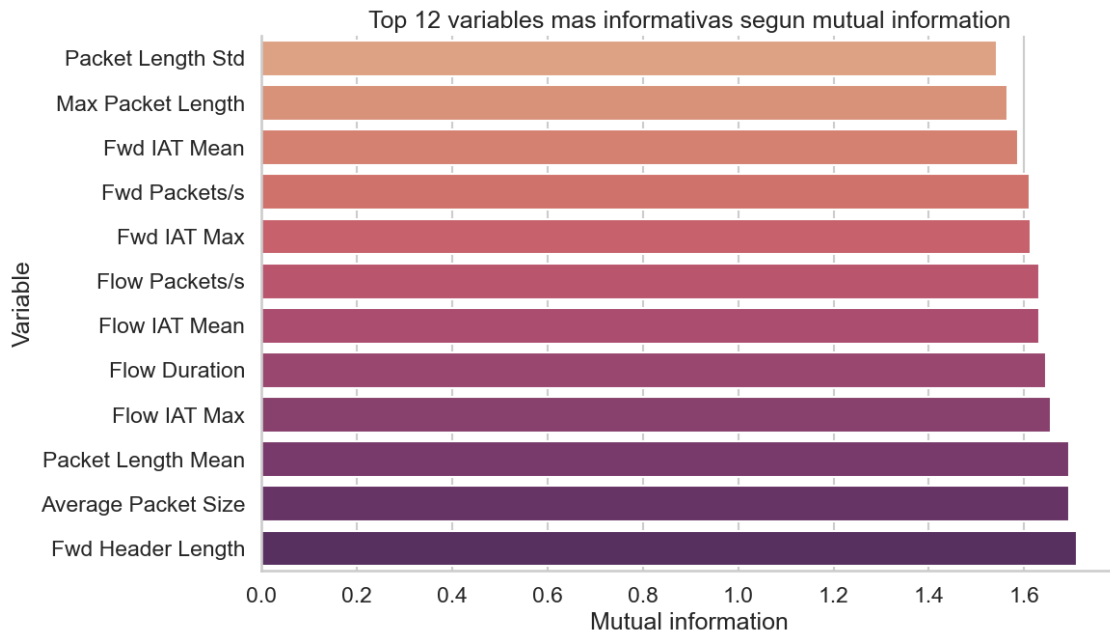
	feature	category \
0	Fwd Header Length	Cabeceras y control
1	Average Packet Size	Tamaño de paquetes
2	Packet Length Mean	Tamaño de paquetes
3	Flow IAT Max	Temporalidad
4	Flow Duration	Temporalidad
5	Flow IAT Mean	Temporalidad
6	Flow Packets/s	Volumen
7	Fwd IAT Max	Temporalidad
8	Fwd Packets/s	Volumen
9	Fwd IAT Mean	Temporalidad
10	Max Packet Length	Tamaño de paquetes
11	Packet Length Std	Tamaño de paquetes

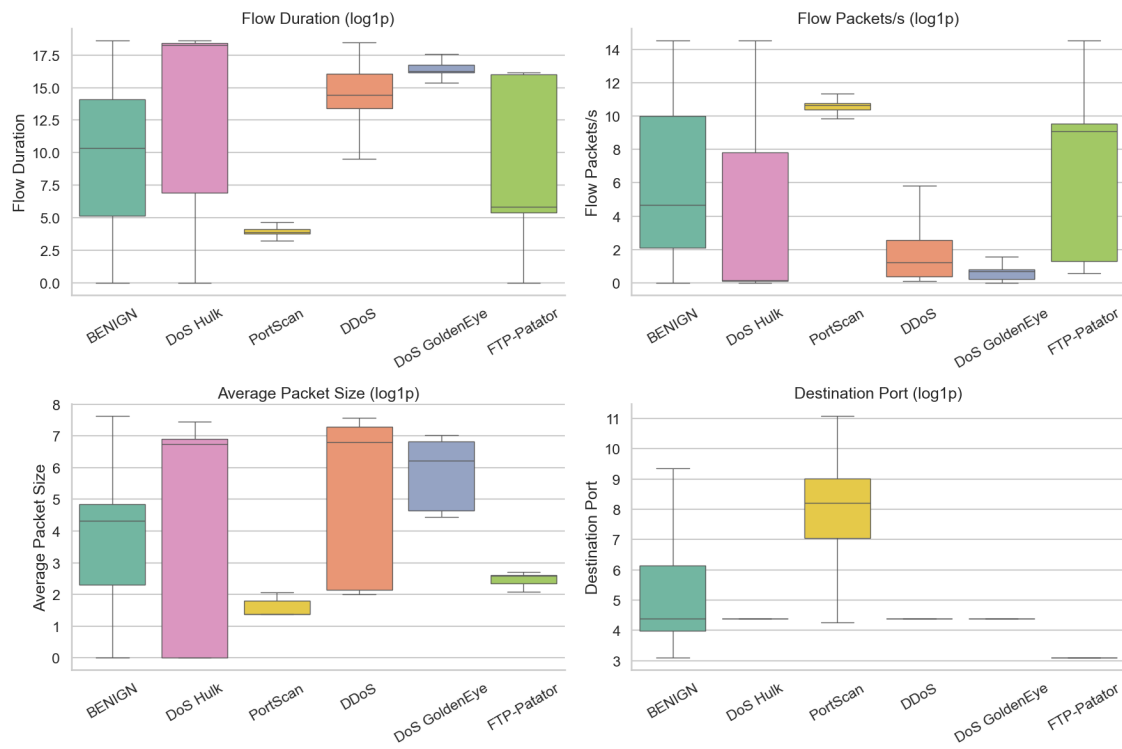
	description \
0	Longitud acumulada de cabeceras en dirección f...
1	Promedio general de bytes por paquete del flujo.
2	Tamaño promedio de los paquetes observados en ...
3	Mayor tiempo entre llegadas dentro del flujo.
4	Duración total del flujo desde el primer hasta...
5	Promedio del tiempo entre llegadas sucesivas d...
6	Tasa de paquetes por segundo del flujo.
7	Máximo tiempo entre llegadas en dirección forw...
8	Paquetes por segundo emitidos en dirección for...
9	Promedio del tiempo entre llegadas en direccio...
10	Mayor tamaño de paquete observado en el flujo.
11	Desviación estándar del tamaño de paquetes del...

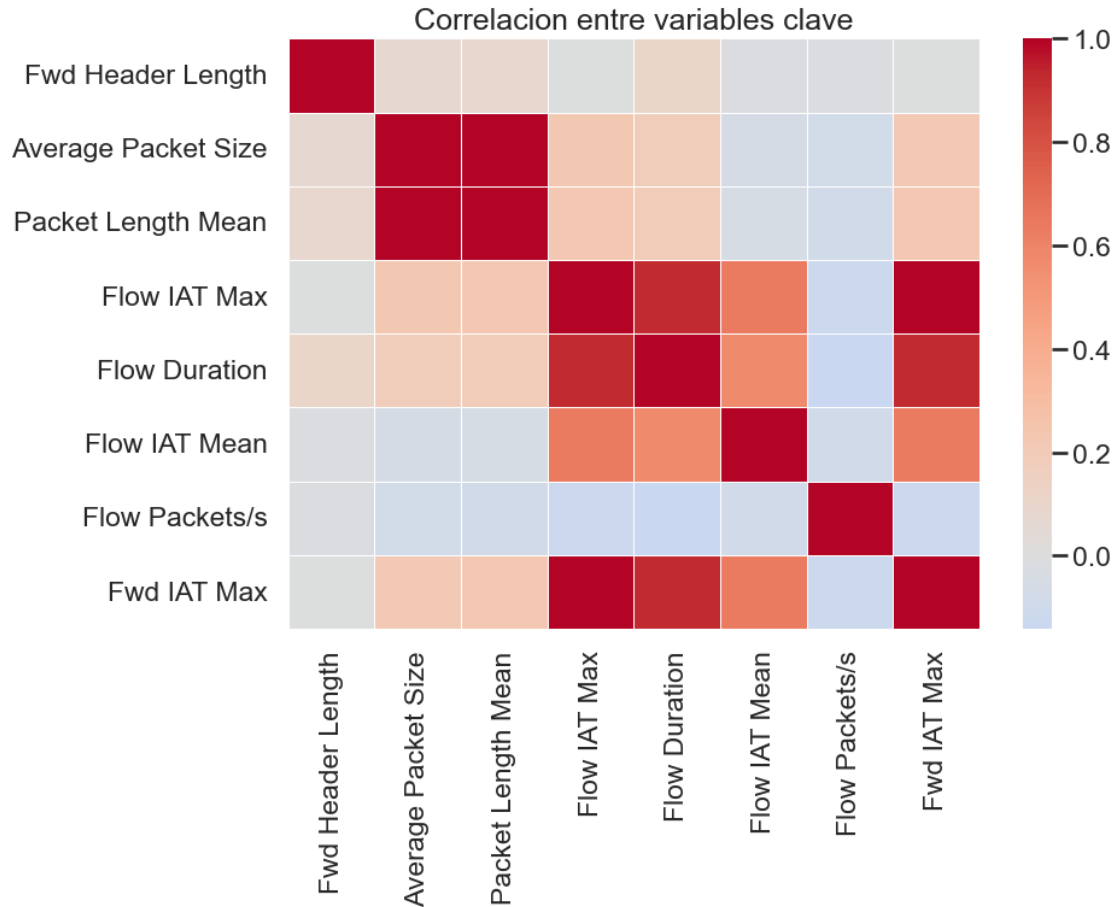
	security_value	mutual_info \
0	Suele correlacionar con la cantidad y tipo de ...	1.7102
1	Resume el perfil de tamaño de la sesión y suel...	1.6943
2	Los ataques automatizados suelen mostrar perfi...	1.6939
3	Ayuda a encontrar flujos con pausas largas o c...	1.6559
4	Separa ataques muy breves y repetitivos, como ...	1.6454
5	Distingue patrones regulares de automatización...	1.6323
6	Es especialmente útil para evidenciar DDoS, Do...	1.6316
7	Resalta pausas largas del emisor, útiles para ...	1.6136
8	Es útil para reconocer rafagas del cliente dur...	1.6119
9	Permite comparar la cadencia con la que el ori...	1.5873
10	Permite distinguir flujos con paquetes grandes...	1.5649
11	Mide variabilidad; secuencias muy uniformes pu...	1.5426

	attack_vs_benign_shift
0	0.0185
1	0.1550
2	0.1379

3	0.0822
4	0.1449
5	0.0369
6	0.0005
7	0.0802
8	0.0003
9	0.0389
10	0.0303
11	0.0268







2.4.1 Hallazgos principales

- El conjunto contiene **2,830,743 registros**, **78 variables** y **15 clases** distribuidas en **8 archivos** diarios.
- Existe un **desbalance fuerte de clases**: las mas frecuentes son BENIGN (80.30%), DoS Hulk (8.16%), PortScan (5.61%), DDoS (4.52%). Esto implica que metricas de accuracy por si solas serian enganosas en etapas de modelado.
- La estructura temporal del dataset es muy util para el contexto: **Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv** es el archivo con mayor proporcion de ataques (56.71%), mientras que **Wednesday-workingHours.pcap_ISCX.csv** concentra la mayor diversidad de clases (6 clases).
- Los principales problemas de calidad estan concentrados en **Flow Packets/s**, **Flow Bytes/s**, con infinitos generados por divisiones sobre tasas, ademas de **256,479 filas duplicadas** dentro de archivos.
- Se detectaron **8 columnas constantes** y **3 pares de columnas redundantes exactas**, por lo que conviene excluirlas antes de entrenar modelos.
- Las variables con mas señal combinan **Cabeceras y control**, **Tamano de paquetes**, **Temporalidad**, **Volumen**. En especial destacan medidas de duracion, tasas por segundo, tamanos de paquete y longitud de cabeceras en direccion forward.

- Para la narrativa del informe, una buena idea es explicar las variables clave por familias: **volumen, temporalidad, tamaño de paquetes, simetría del flujo y banderas TCP.**

```
[3]: # Tabla de características clave para el reporte
feature_report = (
    artifacts["key_features"][["feature", "category", "description",
↪ "security_value", "mutual_info", "attack_vs_benign_shift"]]
    .rename(columns={
        "feature": "Caracteristica",
        "category": "Categoria",
        "description": "Descripcion",
        "security_value": "Valor para IDS",
        "mutual_info": "Mutual information",
        "attack_vs_benign_shift": "Cambio ataque vs benigno",
    })
)

feature_report
```

```
[3]:
```

	Caracteristica	Categoria \
0	Fwd Header Length	Cabeceras y control
1	Average Packet Size	Tamaño de paquetes
2	Packet Length Mean	Tamaño de paquetes
3	Flow IAT Max	Temporalidad
4	Flow Duration	Temporalidad
5	Flow IAT Mean	Temporalidad
6	Flow Packets/s	Volumen
7	Fwd IAT Max	Temporalidad
8	Fwd Packets/s	Volumen
9	Fwd IAT Mean	Temporalidad
10	Max Packet Length	Tamaño de paquetes
11	Packet Length Std	Tamaño de paquetes

	Descripcion \
0	Longitud acumulada de cabeceras en direccion f...
1	Promedio general de bytes por paquete del flujo.
2	Tamaño promedio de los paquetes observados en ...
3	Mayor tiempo entre llegadas dentro del flujo.
4	Duración total del flujo desde el primer hasta...
5	Promedio del tiempo entre llegadas sucesivas d...
6	Tasa de paquetes por segundo del flujo.
7	Maximo tiempo entre llegadas en direccion forw...
8	Paquetes por segundo emitidos en direccion for...
9	Promedio del tiempo entre llegadas en direccio...
10	Mayor tamaño de paquete observado en el flujo.
11	Desviación estandar del tamaño de paquetes del...

	Valor para IDS	Mutual information \
0	Suele correlacionar con la cantidad y tipo de ...	1.7102
1	Resume el perfil de tamaño de la sesión y suel...	1.6943
2	Los ataques automatizados suelen mostrar perfi...	1.6939
3	Ayuda a encontrar flujos con pausas largas o c...	1.6559
4	Separa ataques muy breves y repetitivos, como ...	1.6454
5	Distingue patrones regulares de automatización...	1.6323
6	Es especialmente útil para evidenciar DDoS, Do...	1.6316
7	Resalta pausas largas del emisor, útiles para ...	1.6136
8	Es útil para reconocer ráfagas del cliente dur...	1.6119
9	Permite comparar la cadencia con la que el ori...	1.5873
10	Permite distinguir flujos con paquetes grandes...	1.5649
11	Mide variabilidad; secuencias muy uniformes pu...	1.5426

	Cambio ataque vs benigno
0	0.0185
1	0.1550
2	0.1379
3	0.0822
4	0.1449
5	0.0369
6	0.0005
7	0.0802
8	0.0003
9	0.0389
10	0.0303
11	0.0268

```
[4]: from IPython.display import Markdown, display

overview = artifacts["overview"].set_index("metric")["value"]
label_distribution = artifacts["label_distribution"].copy()
quality = artifacts["quality"].copy()
duplicates = artifacts["duplicates"].copy()

top_attacks = label_distribution[label_distribution["label"] != "BENIGN"].
    ↪head(3)
top_attacks_text = ", ".join(
    f"{row.label} ({row.pct:.2f}%)"
    for row in top_attacks.itertuples(index=False)
)

rare_classes = label_distribution[label_distribution["pct"] < 0.01]["label"].
    ↪tolist()
rare_classes_text = ", ".join(rare_classes) if rare_classes else "ninguna"

quality_notes = []
```

```

for row in quality.itertuples(index=False):
    parts = []
    if row.missing_values:
        parts.append(f"{int(row.missing_values)} nulos")
    if row.infinite_values:
        parts.append(f"{int(row.infinite_values)} infinitos")
    quality_notes.append(f"{row.feature}: {'; '.join(parts)}")

duplicate_total = int(duplicates["duplicate_rows"].sum())
quality_notes_text = ", ".join(quality_notes) if quality_notes else "sin
    problemas de calidad visibles"

display(Markdown(f"""
## Resumen interpretativo del EDA

**Contexto del dataset.** Segun la documentacion oficial del CIC, el trafico
    fue capturado entre el **3 y el 7 de julio de 2017**. El **lunes** contiene
    solo trafico benigno y entre **martes y viernes** se introducen ataques de
    fuerza bruta, DoS/DDoS, web attacks, botnet, infiltration, heartbleed y port
    scanning.

**Hallazgos principales sobre tu copia del dataset descargada desde Kaggle.**
- El conjunto analizado contiene **{int(overview["Registros totales"]):,}
    registros**, **{int(overview["Caracteristicas numericas y de red"])}
    variables** y **{int(overview["Clases de trafico"])} clases**.
- Existe **desbalance de clases**: `BENIGN` representa **{label_distribution.
    iloc[0]["pct"]:.2f}%** del total. Las familias de ataque mas frecuentes son
    **{top_attacks_text}**.
- Las clases extremadamente raras son **{rare_classes_text}**, por lo que
    conviene tratarlas con cuidado al graficar o entrenar modelos.
- Las variables con mas poder descriptivo se concentran en **duracion del
    flujo**, **tasas por segundo**, **tiempos entre llegadas**, **tamano de
    paquetes** y **longitud de cabeceras**.
- En calidad de datos se observan **{quality_notes_text}** y **{duplicate_total:
    ,} filas duplicadas** dentro de archivos.
- Antes de modelar, conviene **limpiar infinitos**, **eliminar columnas
    constantes o redundantes** y usar una estrategia para el **desbalance de
    clases**.

**Fuentes base para contextualizar el dataset.**
- UNB CIC: https://www.unb.ca/cic/datasets/ids-2017.html
- Kaggle: https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset/
    data
"""))

```

2.5 Resumen interpretativo del EDA

Contexto del dataset. Segun la documentacion oficial del CIC, el trafico fue capturado entre el **3 y el 7 de julio de 2017**. El **lunes** contiene solo trafico benigno y entre **martes y viernes** se introducen ataques de fuerza bruta, DoS/DDoS, web attacks, botnet, infiltration, heartbleed y port scanning.

Hallazgos principales sobre tu copia del dataset descargada desde Kaggle. - El conjunto analizado contiene **2,830,743 registros**, **78 variables** y **15 clases**. - Existe **desbalance de clases**: BENIGN representa **80.30%** del total. Las familias de ataque mas frecuentes son **DoS Hulk (8.16%)**, **PortScan (5.61%)**, **DDoS (4.52%)**. - Las clases extremadamente raras son **Infiltration, Web Attack - Sql Injection, Heartbleed**, por lo que conviene tratarlas con cuidado al graficar o entrenar modelos. - Las variables con mas poder descriptivo se concentran en **duracion del flujo, tasas por segundo, tiempos entre llegadas, tamano de paquetes y longitud de cabeceras**. - En calidad de datos se observan **Flow Packets/s: 2867 infinitos, Flow Bytes/s: 1358 nulos; 1509 infinitos y 256,479 filas duplicadas** dentro de archivos. - Antes de modelar, conviene **limpiar infinitos, eliminar columnas constantes o redundantes** y usar una estrategia para el **desbalance de clases**.

Fuentes base para contextualizar el dataset. - UNB CIC: <https://www.unb.ca/cic/datasets/ids-2017.html> - Kaggle: <https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset/data>

2.6 4. Generación y selección de características a utilizar (Feature Engineering)

En esta sección se implementa la limpieza, codificación de etiquetas, eliminación de columnas constantes/redundantes y la selección del Top 20 de características mediante Mutual Information, listas para futuros modelos.

```
[5]: import pandas as pd
import numpy as np
import warnings
import time
from importlib.util import find_spec
from pathlib import Path
from sklearn.feature_selection import mutual_info_classif
from sklearn.preprocessing import LabelEncoder
from sklearn.utils import resample

warnings.filterwarnings('ignore')

# Configuracion de rutas
BASE_DIR = Path().resolve()
DATASET_DIR = BASE_DIR / "dataset"
OUTPUT_FILE = DATASET_DIR / "cicids2017_procesado.parquet"
OUTPUT_CSV = DATASET_DIR / "cicids2017_procesado.csv"

def get_parquet_engine():
```

```

for engine in ("pyarrow", "fastparquet"):
    if find_spec(engine):
        return engine
return None

def load_and_merge_data(dataset_dir):
    csv_files = sorted(Path(dataset_dir).glob("*.csv"))
    if not csv_files:
        raise FileNotFoundError(f"No se encontraron archivos CSV en {dataset_dir}")

    print(f"Encontrados {len(csv_files)} archivos CSV. Cargando...")
    dfs = []
    for file in csv_files:
        print(f" -> Cargando {file.name}")
        df = pd.read_csv(file)
        # Limpiar nombres de columnas (eliminar espacios extra)
        df.columns = [col.strip() for col in df.columns]
        dfs.append(df)

    merged_df = pd.concat(dfs, ignore_index=True)
    print()
    print(f"Dataset consolidado con {merged_df.shape[0]:,} filas y {merged_df.shape[1]} columnas.")
    return merged_df

def clean_data(df):
    initial_rows = df.shape[0]

    # 2.1 Eliminar duplicados
    print("Eliminando filas duplicadas...")
    df.drop_duplicates(inplace=True)
    new_rows = df.shape[0]
    print(f" -> {initial_rows - new_rows:,} filas duplicadas eliminadas.")

    # 2.2 Reemplazar infinitos por NaN
    print("Manejando infinitos y valores nulos...")
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    df[numeric_cols] = df[numeric_cols].replace([np.inf, -np.inf], np.nan)

    # 2.3 Eliminar filas con NaN
    before_dropna = df.shape[0]
    df.dropna(inplace=True)
    print(f" -> {before_dropna - df.shape[0]:,} filas con NaNs o infinitos eliminadas.")

```

```

print(f"Limpieza completada. Filas actuales: {df.shape[0]:,}")
return df

def encode_labels(df):
    print("Limpiando y codificando etiquetas...")
    df["Label"] = df["Label"].astype(str).str.strip()
    df["Label"] = df["Label"].str.replace(r"Web Attack.*Brute Force", "Web_
↪Attack - Brute Force", regex=True)
    df["Label"] = df["Label"].str.replace(r"Web Attack.*XSS", "Web Attack -_
↪XSS", regex=True)
    df["Label"] = df["Label"].str.replace(r"Web Attack.*Sql Injection", "Web_
↪Attack - Sql Injection", regex=True)

    # Generacion Label_Binary
    df["Label_Binary"] = np.where(df["Label"] == "BENIGN", 0, 1)

    # Generacion Label_Multiclass
    le = LabelEncoder()
    df["Label_Multiclass"] = le.fit_transform(df["Label"])

    class_mapping = dict(zip(le.classes_, le.transform(le.classes_)))
    print()
    print("Mapeo de clases a numeros:")
    for k, v in class_mapping.items():
        print(f" {v} -> {k}")

    return df, le

def remove_redundant_features(df):
    # Columnas constantes
    constant_features = [
        "Bwd PSH Flags", "Bwd URG Flags", "Fwd Avg Bytes/Bulk", "Fwd Avg_
↪Packets/Bulk",
        "Fwd Avg Bulk Rate", "Bwd Avg Bytes/Bulk", "Bwd Avg Packets/Bulk", "Bwd_
↪Avg Bulk Rate"
    ]

    # Columnas redundantes
    redundant_features = [
        "Fwd Header Length.1", # Redundante con Fwd Header Length
        "Subflow Fwd Packets", # Redundante con Total Fwd Packets
        "Subflow Bwd Packets" # Redundante con Total Backward Packets
    ]

```

```

    cols_to_drop = [col for col in constant_features + redundant_features if
↪col in df.columns]
    if cols_to_drop:
        print(f"Eliminando columnas constantes y redundantes: {cols_to_drop}")
        df.drop(columns=cols_to_drop, inplace=True)

    return df

def select_top_features(df, target_col="Label_Binary", n_features=20):
    print()
    print("Generando muestra para calculo de Mutual Information...")
    sample_df = resample(df, n_samples=100000, random_state=42,
↪stratify=df["Label_Multiclass"])

    features = sample_df.drop(columns=["Label", "Label_Binary",
↪"Label_Multiclass"])
    target = sample_df[target_col]

    print("Calculando Mutual Information...")
    start_mi = time.time()
    mi_scores = mutual_info_classif(features, target, random_state=42)
    print(f"MI calculado en {time.time() - start_mi:.2f} segundos")

    mi_df = pd.DataFrame({"Feature": features.columns, "Mutual_Info":
↪mi_scores})
    mi_df = mi_df.sort_values(by="Mutual_Info", ascending=False).
↪reset_index(drop=True)

    top_features = mi_df.head(40)["Feature"].tolist()
    print()
    print("Top características basadas en Mutual Information:")
    print(mi_df.head(10))

    # Filtrado por correlacion
    print()
    print("Filtrando características altamente correlacionadas (>0.95)...")
    corr_matrix = df[top_features].corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).
↪astype(bool))
    to_drop_corr = [column for column in upper.columns if any(upper[column] > 0.
↪95)]

    print(f"Características eliminadas por superponerse: {to_drop_corr}")

    final_features = [f for f in top_features if f not in to_drop_corr]

```

```

final_features = final_features[:n_features]

print()
print(f"Top {len(final_features)} características finales seleccionadas:")
for f in final_features:
    print(f" - {f}")

return final_features

def main():
    start_time = time.time()

    # 1. Cargar
    df = load_and_merge_data(DATASET_DIR)

    # 2. Limpiar
    df = clean_data(df)

    # 3. Label Encoder
    df, le = encode_labels(df)

    # 4. Remover redundantes
    df = remove_redundant_features(df)

    # 5. Seleccionar top features
    final_features = select_top_features(df, n_features=20)

    # 6. Exportar
    cols_to_keep = final_features + ["Label", "Label_Binary",
↪ "Label_Multiclass"]
    print()
    print(f"Extrayendo subset optimizado ({len(cols_to_keep)} columnas)...")
    df_final = df[cols_to_keep].copy()

    # Optimizacion de tipos de datos para parquet / memoria DL
    for col in df_final.columns:
        if col == "Label":
            continue
        if df_final[col].dtype == "float64":
            df_final[col] = df_final[col].astype("float32")
        elif df_final[col].dtype == "int64" and df_final[col].max() <
↪ 2147483647:
            df_final[col] = df_final[col].astype("int32")

    parquet_engine = get_parquet_engine()
    if parquet_engine:

```

```

        print(f"Guardando archivo en formato Parquet con_
↪engine='{parquet_engine}'...")
        df_final.to_parquet(OUTPUT_FILE, index=False, engine=parquet_engine)
        print(f"Archivo guardado exitosamente: {OUTPUT_FILE}")
    else:
        print("No se encontro un motor Parquet instalado ('pyarrow' o_
↪'fastparquet').")
        print("Guardando respaldo en CSV para no interrumpir el flujo...")
        df_final.to_csv(OUTPUT_CSV, index=False)
        print(f"Archivo CSV guardado exitosamente: {OUTPUT_CSV}")

    print()
    print(f"Proceso finalizado en {time.time() - start_time:.2f} segundos.")

if __name__ == '__main__':
    main()

```

Encontrados 8 archivos CSV. Cargando...

```

-> Cargando Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv
-> Cargando Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv
-> Cargando Friday-WorkingHours-Morning.pcap_ISCX.csv
-> Cargando Monday-WorkingHours.pcap_ISCX.csv
-> Cargando Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv
-> Cargando Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv
-> Cargando Tuesday-WorkingHours.pcap_ISCX.csv
-> Cargando Wednesday-workingHours.pcap_ISCX.csv

```

Dataset consolidado con 2,830,743 filas y 79 columnas.

Eliminando filas duplicadas...

```

-> 308,381 filas duplicadas eliminadas.

```

Manejando infinitos y valores nulos...

```

-> 1,564 filas con NaNs o infinitos eliminadas.

```

Limpieza completada. Filas actuales: 2,520,798

Limpiando y codificando etiquetas...

Mapeo de clases a numeros:

```

0 -> BENIGN
1 -> Bot
2 -> DDoS
3 -> DoS GoldenEye
4 -> DoS Hulk

```

```

5 -> DoS Slowhttptest
6 -> DoS slowloris
7 -> FTP-Patator
8 -> Heartbleed
9 -> Infiltration
10 -> PortScan
11 -> SSH-Patator
12 -> Web Attack - Brute Force
13 -> Web Attack - Sql Injection
14 -> Web Attack - XSS

```

Eliminando columnas constantes y redundantes: ['Bwd PSH Flags', 'Bwd URG Flags', 'Fwd Avg Bytes/Bulk', 'Fwd Avg Packets/Bulk', 'Fwd Avg Bulk Rate', 'Bwd Avg Bytes/Bulk', 'Bwd Avg Packets/Bulk', 'Bwd Avg Bulk Rate', 'Fwd Header Length.1', 'Subflow Fwd Packets', 'Subflow Bwd Packets']

Generando muestra para calculo de Mutual Information...

Calculando Mutual Information...

MI calculado en 28.06 segundos

Top características basadas en Mutual Information:

	Feature	Mutual_Info
0	Average Packet Size	0.337017
1	Packet Length Std	0.326702
2	Packet Length Variance	0.326588
3	Packet Length Mean	0.311172
4	Subflow Bwd Bytes	0.298564
5	Total Length of Bwd Packets	0.298322
6	Avg Bwd Segment Size	0.291803
7	Bwd Packet Length Mean	0.289632
8	Max Packet Length	0.273478
9	Total Length of Fwd Packets	0.267805

Filtrando características altamente correlacionadas (>0.95)...

Características eliminadas por superponerse: ['Packet Length Mean', 'Total Length of Bwd Packets', 'Bwd Packet Length Mean', 'Max Packet Length', 'Subflow Fwd Bytes', 'Bwd Packet Length Max', 'Bwd Packet Length Std', 'Flow IAT Max', 'Avg Fwd Segment Size', 'Flow Duration', 'Fwd Packet Length Std', 'Flow Packets/s']

Top 20 características finales seleccionadas:

- Average Packet Size
- Packet Length Std
- Packet Length Variance
- Subflow Bwd Bytes
- Avg Bwd Segment Size
- Total Length of Fwd Packets

- Init_Win_bytes_backward
- Destination Port
- Init_Win_bytes_forward
- Fwd Packet Length Max
- Fwd IAT Max
- Fwd IAT Total
- Fwd IAT Mean
- Fwd Packet Length Mean
- Flow Bytes/s
- Fwd Header Length
- Bwd Packets/s
- Bwd Header Length
- Fwd Packets/s
- Fwd IAT Std

Extrayendo subset optimizado (23 columnas)...

Guardando archivo en formato Parquet con engine='pyarrow'...

Archivo guardado exitosamente: C:\Users\javid\OneDrive\Documentos\U\Data Science\Intrusion_Detection_System\dataset\cicids2017_procesado.parquet

Proceso finalizado en 88.27 segundos.

3 Fase 2

Objetivo: Implementar y evaluar el rendimiento de un modelo que resuelva el problema investigado

3.1 5. Modelo 1: Decision Tree para deteccion de trafico benigno y ataques

En esta fase se entrena un primer modelo de machine learning para el problema IDS usando el dataset limpio `cicids2017_procesado.parquet`. La variable objetivo sera `Label`, por lo que el arbol debe distinguir trafico `BENIGN` y cada una de las familias de ataque presentes en `CICIDS2017`.

El flujo seguira cuatro pasos: cargar el dataset limpio, dividirlo en conjuntos de entrenamiento, validacion y prueba, seleccionar la mejor configuracion del arbol con validacion y finalmente evaluar el modelo con metricas multiclase y curvas ROC one-vs-rest.

```
[6]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from sklearn.metrics import accuracy_score, classification_report, \
    precision_recall_fscore_support
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

sns.set_theme(style='whitegrid')
```

```

DATASET_PATH = Path('dataset') / 'cicids2017_procesado.parquet'
df_model = pd.read_parquet(DATASET_PATH)

X = df_model.drop(columns=['Label', 'Label_Binary', 'Label_Multiclass'])
y = df_model['Label']

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)

split_summary = pd.DataFrame({
    'split': ['train', 'validation', 'test'],
    'rows': [len(X_train), len(X_val), len(X_test)],
})
split_summary['pct'] = split_summary['rows'] / len(df_model)

class_summary = pd.concat([
    y_train.value_counts(normalize=True).rename('train_pct'),
    y_val.value_counts(normalize=True).rename('validation_pct'),
    y_test.value_counts(normalize=True).rename('test_pct'),
    y.value_counts().rename('total_rows')
], axis=1).fillna(0)
class_summary = class_summary.sort_values('total_rows', ascending=False)

display(split_summary)
display(class_summary.head(15))

```

	split	rows	pct
0	train	1764558	0.70
1	validation	378120	0.15
2	test	378120	0.15

	train_pct	validation_pct	test_pct	total_rows
Label				
BENIGN	0.831108	0.831109	0.831109	2095057
DoS Hulk	0.068568	0.068568	0.068568	172846
DDoS	0.050783	0.050783	0.050783	128014
PortScan	0.035978	0.035978	0.035978	90694
DoS GoldenEye	0.004080	0.004081	0.004081	10286
FTP-Patator	0.002353	0.002351	0.002354	5931
DoS slowloris	0.002136	0.002137	0.002137	5385
DoS Slowhttptest	0.002074	0.002073	0.002073	5228
SSH-Patator	0.001277	0.001277	0.001277	3219
Bot	0.000773	0.000772	0.000772	1948

Web Attack - Brute Force	0.000583	0.000584	0.000582	1470
Web Attack - XSS	0.000258	0.000259	0.000259	652
Infiltration	0.000014	0.000013	0.000016	36
Web Attack - Sql Injection	0.000009	0.000008	0.000008	21
Heartbleed	0.000005	0.000005	0.000003	11

3.1.1 Interpretacion del split de datos

El conjunto limpio contiene **2,520,798 registros** y fue dividido en una proporcion **70% entrenamiento, 15% validacion y 15% prueba**. Eso deja **1,764,558 filas** para entrenar, **378,120** para validar decisiones de modelado y **378,120** para la evaluacion final.

La tabla de distribucion confirma un punto clave del problema IDS: el dataset esta **fuertemente desbalanceado**. BENIGN domina el volumen y algunas clases como Heartbleed, Infiltration y Web Attack - Sql Injection aparecen muy poco. Por eso no basta con mirar solo accuracy; tambien necesitaremos metricas macro y por clase, porque en seguridad importa detectar familias raras sin quedar cegados por la clase mayoritaria.

```
[7]: candidate_params = [
    {'criterion': 'gini', 'max_depth': 8, 'min_samples_leaf': 5},
    {'criterion': 'gini', 'max_depth': 12, 'min_samples_leaf': 5},
    {'criterion': 'gini', 'max_depth': 16, 'min_samples_leaf': 10},
    {'criterion': 'entropy', 'max_depth': 12, 'min_samples_leaf': 5},
    {'criterion': 'entropy', 'max_depth': 16, 'min_samples_leaf': 10},
    {'criterion': 'log_loss', 'max_depth': 16, 'min_samples_leaf': 10},
]

validation_rows = []
for params in candidate_params:
    model = DecisionTreeClassifier(random_state=42, class_weight='balanced',
    ↪**params)
    model.fit(X_train, y_train)
    y_val_pred = model.predict(X_val)
    precision_macro, recall_macro, f1_macro, _ =
    ↪precision_recall_fscore_support(
        y_val, y_val_pred, average='macro', zero_division=0
    )
    precision_weighted, recall_weighted, f1_weighted, _ =
    ↪precision_recall_fscore_support(
        y_val, y_val_pred, average='weighted', zero_division=0
    )
    validation_rows.append({
        **params,
        'accuracy': accuracy_score(y_val, y_val_pred),
        'precision_macro': precision_macro,
        'recall_macro': recall_macro,
        'f1_macro': f1_macro,
        'precision_weighted': precision_weighted,
```

```

        'recall_weighted': recall_weighted,
        'f1_weighted': f1_weighted,
    })

validation_results = pd.DataFrame(validation_rows).sort_values(
    by=['f1_macro', 'f1_weighted', 'accuracy'], ascending=False
).reset_index(drop=True)

display(validation_results.round(4))

```

	criterion	max_depth	min_samples_leaf	accuracy	precision_macro	\
0	entropy	16	10	0.9969	0.7855	
1	log_loss	16	10	0.9969	0.7855	
2	entropy	12	5	0.9927	0.7496	
3	gini	16	10	0.9876	0.6783	
4	gini	12	5	0.9641	0.6358	
5	gini	8	5	0.8215	0.5235	

	recall_macro	f1_macro	precision_weighted	recall_weighted	f1_weighted
0	0.9036	0.8237	0.9979	0.9969	0.9973
1	0.9036	0.8237	0.9979	0.9969	0.9973
2	0.9045	0.7851	0.9962	0.9927	0.9942
3	0.8669	0.7201	0.9941	0.9876	0.9905
4	0.8593	0.6584	0.9840	0.9641	0.9722
5	0.7752	0.5114	0.9855	0.8215	0.8923

3.1.2 Interpretacion de la validacion

La mejor configuracion en validacion fue el arbol con `criterion='entropy'`, `max_depth=16` y `min_samples_leaf=10`, junto con `class_weight='balanced'`. Ese modelo alcanzo aproximadamente **0.9969 de accuracy** y **0.8237 de F1 macro**, superando al resto especialmente en la metrica macro.

Esto importa porque **F1 macro** da el mismo peso a todas las clases, incluidas las minoritarias. En un IDS eso es preferible a seleccionar el modelo solo por **accuracy**, ya que un clasificador puede acertar casi todo el trafico benigno y aun asi fallar donde mas interesa: los ataques poco frecuentes. Por esa razon, el arbol elegido se entrenara de nuevo usando `train + validation` antes de medir su desempeno final en `test`.

```

[8]: best_params = validation_results.loc[0, ['criterion', 'max_depth',
      ↪ 'min_samples_leaf']].to_dict()
best_params['max_depth'] = int(best_params['max_depth'])
best_params['min_samples_leaf'] = int(best_params['min_samples_leaf'])

X_trainval = pd.concat([X_train, X_val], axis=0)
y_trainval = pd.concat([y_train, y_val], axis=0)

decision_tree = DecisionTreeClassifier(

```

```

    random_state=42,
    class_weight='balanced',
    **best_params
)
decision_tree.fit(X_trainval, y_trainval)

y_test_pred = decision_tree.predict(X_test)
y_test_proba = decision_tree.predict_proba(X_test)

overall_metrics = pd.DataFrame([
    {
        'accuracy': accuracy_score(y_test, y_test_pred),
        'precision_macro': precision_recall_fscore_support(y_test, y_test_pred,
↪average='macro', zero_division=0)[0],
        'recall_macro': precision_recall_fscore_support(y_test, y_test_pred,
↪average='macro', zero_division=0)[1],
        'f1_macro': precision_recall_fscore_support(y_test, y_test_pred,
↪average='macro', zero_division=0)[2],
        'precision_weighted': precision_recall_fscore_support(y_test,
↪y_test_pred, average='weighted', zero_division=0)[0],
        'recall_weighted': precision_recall_fscore_support(y_test, y_test_pred,
↪average='weighted', zero_division=0)[1],
        'f1_weighted': precision_recall_fscore_support(y_test, y_test_pred,
↪average='weighted', zero_division=0)[2],
    }
]).round(4)

report = classification_report(y_test, y_test_pred, output_dict=True,
↪zero_division=0)
report_df = pd.DataFrame(report).T.reset_index().rename(columns={'index':
↪'label'})
report_df = report_df[report_df['label'].isin(decision_tree.classes_)]
report_df = report_df[['label', 'precision', 'recall', 'f1-score', 'support']]
report_df = report_df.sort_values('support', ascending=False).
↪reset_index(drop=True)

display(overall_metrics)
display(report_df.round(4))

```

	accuracy	precision_macro	recall_macro	f1_macro	precision_weighted	\
0	0.9966	0.8133	0.9286	0.8464	0.9981	

	recall_weighted	f1_weighted
0	0.9966	0.9972

	label	precision	recall	f1-score	support
0	BENIGN	1.0000	0.9965	0.9982	314259.0

1	DoS Hulk	0.9952	0.9995	0.9973	25927.0
2	DDoS	0.9980	0.9997	0.9989	19202.0
3	PortScan	0.9886	0.9993	0.9939	13604.0
4	DoS GoldenEye	0.9866	0.9987	0.9926	1543.0
5	FTP-Patator	0.9989	1.0000	0.9994	890.0
6	DoS slowloris	0.9745	0.9938	0.9841	808.0
7	DoS Slowhttptest	0.9642	0.9949	0.9793	784.0
8	SSH-Patator	1.0000	1.0000	1.0000	483.0
9	Bot	0.4887	0.9658	0.6490	292.0
10	Web Attack - Brute Force	0.5857	0.5591	0.5721	220.0
11	Web Attack - XSS	0.1355	0.7551	0.2298	98.0
12	Infiltration	0.7500	1.0000	0.8571	6.0
13	Web Attack - Sql Injection	0.3333	0.6667	0.4444	3.0
14	Heartbleed	1.0000	1.0000	1.0000	1.0

3.1.3 Interpretacion de las metricas de rendimiento

En el conjunto de prueba, el Decision Tree obtuvo aproximadamente **0.9966 de accuracy** y **0.9972 de F1 weighted**, lo que indica un desempeno global muy alto sobre el volumen total del trafico. Sin embargo, la lectura correcta para un IDS aparece en las metricas macro: **precision macro = 0.8133**, **recall macro = 0.9286** y **F1 macro = 0.8464**. Estas cifras son mas bajas porque revelan la dificultad de mantener un rendimiento uniforme en clases raras.

Al revisar la tabla por clase se observa que el arbol detecta muy bien ataques frecuentes como DDoS, DoS Hulk, PortScan, FTP-Patator y SSH-Patator, con valores cercanos a 1.0. En cambio, las familias web presentan mas fragilidad: Web Attack - XSS tiene **precision ~0.1355** y **F1 ~0.2298**, mientras Web Attack - Brute Force queda en **recall ~0.5591**.

En contexto de ciberseguridad, esto significa que el modelo funciona muy bien como **baseline** para separar trafico normal y varios ataques masivos, pero todavia no es suficiente como respuesta final si el objetivo principal es cubrir con confiabilidad los ataques menos frecuentes o mas sutiles. La fuerte diferencia entre metricas weighted y macro confirma que el desbalance sigue influyendo bastante en el resultado.

```
[9]: from sklearn.metrics import auc, roc_curve, roc_auc_score
    from sklearn.preprocessing import label_binarize

    class_names = decision_tree.classes_
    y_test_bin = label_binarize(y_test, classes=class_names)

    roc_rows = []
    fpr = {}
    tpr = {}
    roc_auc = {}

    for idx, class_name in enumerate(class_names):
        fpr[class_name], tpr[class_name], _ = roc_curve(y_test_bin[:, idx],
        ↪ y_test_proba[:, idx])
        roc_auc[class_name] = auc(fpr[class_name], tpr[class_name])
```

```

roc_rows.append({'class': class_name, 'roc_auc': roc_auc[class_name]})

roc_auc_macro = roc_auc_score(y_test_bin, y_test_proba, multi_class='ovr',
    ↪average='macro')
roc_auc_weighted = roc_auc_score(y_test_bin, y_test_proba, multi_class='ovr',
    ↪average='weighted')
roc_auc_micro = roc_auc_score(y_test_bin, y_test_proba, average='micro')

fpr['micro'], tpr['micro'], _ = roc_curve(y_test_bin.ravel(), y_test_proba.
    ↪ravel())
roc_table = pd.DataFrame(roc_rows).sort_values('roc_auc').reset_index(drop=True)
display(roc_table.round(4))

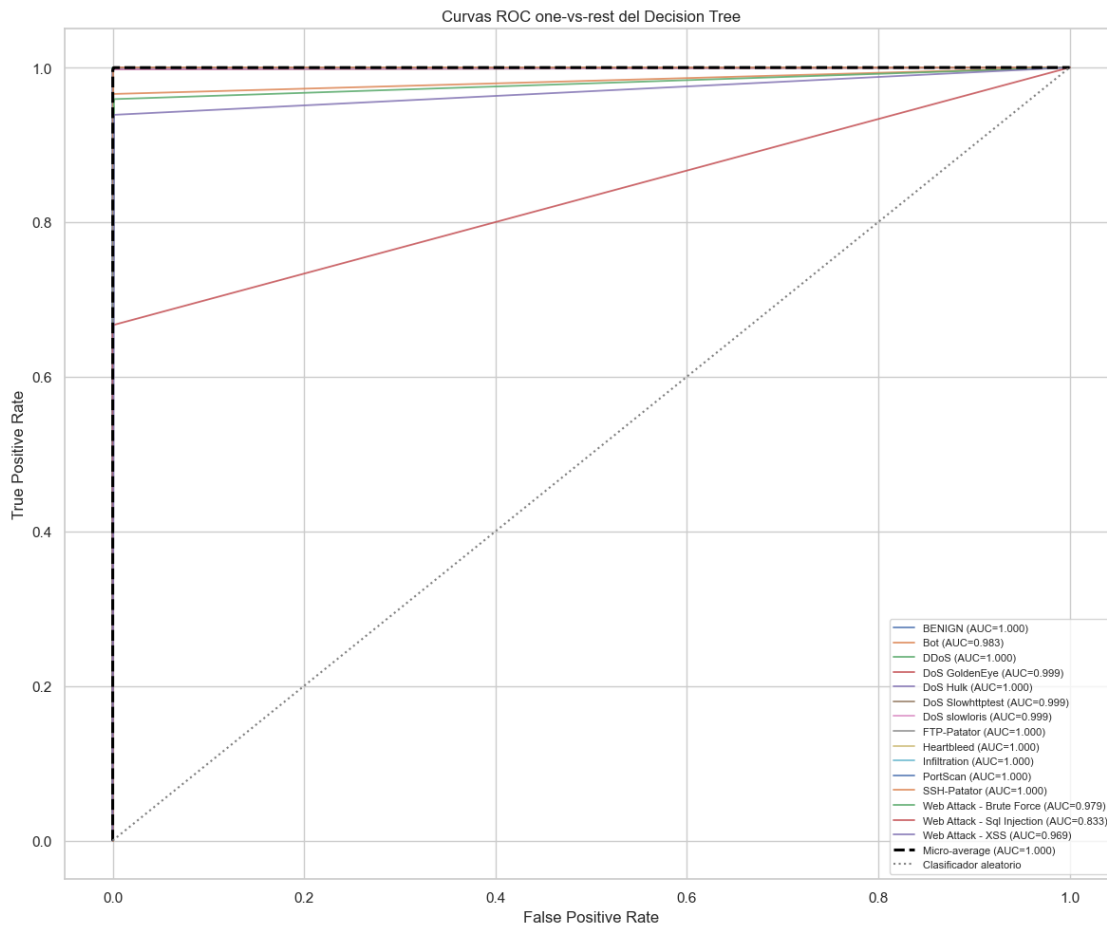
plt.figure(figsize=(12, 10))
for class_name in class_names:
    plt.plot(fpr[class_name], tpr[class_name], linewidth=1.3, alpha=0.85,
    ↪label=f'{class_name} (AUC={roc_auc[class_name]:.3f})')
plt.plot(fpr['micro'], tpr['micro'], linestyle='--', linewidth=2.2,
    ↪color='black', label=f'Micro-average (AUC={roc_auc_micro:.3f})')
plt.plot([0, 1], [0, 1], linestyle=':', color='gray', label='Clasificador
    ↪aleatorio')
plt.title('Curvas ROC one-vs-rest del Decision Tree')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right', fontsize=8)
plt.tight_layout()
plt.show()

pd.DataFrame([{'roc_auc_macro_ovr': roc_auc_macro,
    'roc_auc_weighted_ovr': roc_auc_weighted,
    'roc_auc_micro_ovr': roc_auc_micro,
}]).round(4)

```

	class	roc_auc
0	Web Attack - Sql Injection	0.8333
1	Web Attack - XSS	0.9686
2	Web Attack - Brute Force	0.9791
3	Bot	0.9828
4	DoS Slowhttptest	0.9987
5	DoS slowloris	0.9988
6	DoS GoldenEye	0.9993
7	PortScan	0.9997
8	BENIGN	0.9998
9	DoS Hulk	1.0000
10	DDoS	1.0000
11	Infiltration	1.0000

12	FTP-Patator	1.0000
13	Heartbleed	1.0000
14	SSH-Patator	1.0000



```
[9]:      roc_auc_macro_ovr  roc_auc_weighted_ovr  roc_auc_micro_ovr
0              0.984              0.9997              0.9999
```

3.1.4 Interpretacion de la curva ROC

La evaluacion ROC confirma el mismo patron visto en precision, recall y F1. A nivel agregado, el modelo logra valores muy altos: **ROC AUC macro OVR ~0.9840**, **ROC AUC weighted OVR ~0.9997** y **ROC AUC micro OVR ~0.9999**. Esto significa que, en promedio, el arbol separa bastante bien cada clase frente al resto.

Aun asi, la tabla por clase muestra que no todas las categorias son igual de faciles. **Web Attack - Sql Injection** es la mas debil con **AUC 0.8333**, seguida por **Web Attack - XSS (0.9686)**, **Web Attack - Brute Force (0.9791)** y **Bot (0.9828)**. El resto de ataques queda muy cerca de 1.0, lo cual indica una separacion excelente.

En terminos del problema investigado, el Decision Tree ya demuestra que puede ser un candidato

fuerte para un IDS cuando los ataques tienen patrones bien definidos, pero tambien deja claro que no todas las familias quedan cubiertas con la misma robustez. Por eso, al comparar modelos mas adelante, sera importante priorizar aquellos que mantengan buen rendimiento en las clases minoritarias y no solo una accuracy global alta.

3.1.5 Conclusion parcial sobre este primer modelo

Como primer modelo de machine learning para el problema IDS, el **Decision Tree** ofrece una base muy competitiva: entrena rapido, es interpretable y obtiene resultados sobresalientes en la mayoria de las clases. Sin embargo, sus debilidades en ataques web raros muestran que todavia no se puede declarar como el mejor modelo definitivo solo con este experimento.

La conclusion metodologica correcta es que el arbol funciona muy bien como **baseline de referencia**. En las siguientes comparaciones debera medirse contra modelos que puedan generalizar mejor en clases escasas, para responder con evidencia cual es realmente el mejor modelo para detectar intrusiones en este dataset.